TALENTSYNC: AN AI-POWERED SEMANTIC RESUME-JOB MATCHING AND RECRUITMENT AUTOMATION PLATFORM

A PROJECT REPORT

SUBMITTED BY

SHREYAN GHOSH
ROLL NO-10930622019, REG NO: 221090110067, OF 2022-2023

SAYAN KHAN
ROLL NO-10930622024, REG NO: 221090110064, OF 2022-2023

SWARNIM SINGH
ROLL NO-10930622038, REG NO: 221090110074, OF 2022-2023

ABHISHEK KUMAR
ROLL NO-10930622018, REG NO: 221090110020, OF 2022-2023

RAJAT KUMAR JHA
ROLL NO-10930622017, REG NO: 221090110054, OF 2022-2023

AKASH KUMAR
ROLL NO-10930622060, REG NO: 221090110022, OF 2022-2023

Under the Supervision of
Kallol Bhattacharya

Department of Artificial Intelligence and Machine Learning
Netaji Subhash Engineering College

In partial fulfilment for the award of the degree
Of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

NETAJI SUBHASH ENGINEERING COLLEGE

under

Maulana Abul Kalam Azad University of Technology

West Bengal

July 2026

NETAJI SUBHASH ENGINEERING COLLEGE

under

Maulana Abul Kalam Azad University of Technology

West Bengal

BONAFIDE CERTIFICATE

This is to certify that the project report titled "TalentSync: An AI-Powered Semantic Resume-Job Matching and Recruitment Automation Platform" submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology (B. Tech) in Artificial Intelligence and Machine Learning, under the Maulana Abul Kalam Azad University of Technology (MAKAUT), is a faithful and original record of the work carried out by:

Shreyan Ghosh, Roll No. 10930622019, Regd. No. 221090110067 Of 2022-23

Sayan Khan, Roll No. 10930622024, Regd. No. 221090110064 Of 2022-23

Swarnim Singh, Roll No. 10930622038, Regd. No. 221090110074 Of 2022-23

Abhishek Kumar, Roll No. 10930622018, Regd. No. 221090110020 Of 2022-23

Rajat Kumar Jha, Roll No. 10930622017, Regd. No. 221090110054 Of 2022-23

Akash Kumar, Roll No. 10930622060, Regd. No. 221090110022 Of 2022-23

The project work has been conducted under the supervision and guidance of Kallol Bhattacharya, advisor, Artificial Intelligence & Machine Learning, Netaji Subhash Engineering College, during the academic year 2025-26.

It is further certified that this work is original and has not been submitted previously for the award of any degree or diploma in any university or institution. Any material borrowed from other sources has been duly acknowledged.

We wish the students success in all their future academic and professional endeavours.

Guide's Signature Kallol Bhattacharya Advisor Artificial Intelligence & Machine Learning	HOD's Signature Anupam Ghosh Head of Department Artificial Intelligence & Machine Learning

CANDIDATE'S DECLARATION

We hereby declare that the project report titled “TalentSync: An AI-Powered Semantic Resume-Job Matching and Recruitment Automation Platform” submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology (B.Tech) in Artificial Intelligence and Machine Learning, at Netaji Subhash Engineering College, affiliated to Maulana Abul Kalam Azad University of Technology (MAKAUT), is the result of our original work carried out collectively as a team of final-year undergraduate students.

This work has been conducted under the supervision and guidance of Kallol Bhattacharya, advisor, Artificial Intelligence & Machine Learning, Netaji Subhash Engineering College. We affirm that this report has not been submitted, either wholly or in part, for the award of any other degree, diploma, or similar title at this or any other institution.

We confirm that all sources of information and data used in the preparation of this report have been fully and properly acknowledged. The content of this report represents our joint efforts, and we have each contributed responsibly and ethically to the project. We also affirm that we have complied with the rules and regulations of the institution concerning plagiarism and academic integrity, and that this project is the outcome of our independent and collaborative work.

Shreyan Ghosh < **10930622019**> << **221090110067**>> _____

Sayan Khan < **10930622024**> << **221090110064**>> _____

Swarnim Singh < **10930622038**> << **221090110074**>> _____

Abhishek Kumar < **10930622018**> << **221090110020**>> _____

Rajat Kumar Jha < **10930622017**> << **221090110054**>> _____

Akash Kumar < **10930622060**> << **221090110022**>> _____

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who have supported and guided us throughout the successful completion of our final year project titled "TalentSync: An AI-Powered Semantic Resume-Job Matching and Recruitment Automation Platform".

First and foremost, we extend our heartfelt thanks to Kallol Bhattacharya, our project guide, for his invaluable guidance, continuous encouragement, and expert advice. His profound knowledge and constructive suggestions have been instrumental in shaping our project. We are truly grateful for the time, effort, and support he/she has extended to us during every phase of this work.

We also wish to convey our deep appreciation to Anupam Ghosh, Head of the Department, Artificial Intelligence & Machine Learning, for his constant support, motivation, and for providing the necessary infrastructure and resources that facilitated our project development.

Our sincere thanks also go to all the faculty members of the Artificial Intelligence & Machine Learning for their insightful lectures, seminars, and guidance throughout our academic journey. Their collective knowledge and mentorship have been a foundation upon which this work has been built.

We are also thankful to our fellow classmates and friends, especially those who offered their support, shared ideas, and provided encouragement during the course of the project.

Finally, we are deeply grateful to our families for their unconditional love, patience, and encouragement throughout this endeavour. Their constant moral support kept us motivated and focused.

We sincerely acknowledge the efforts of all those who directly or indirectly contributed to the successful completion of this project.

Shreyan Ghosh < **10930622019**> << **221090110067**>> _____

Sayan Khan < **10930622024**> << **221090110064**>> _____

Swarnim Singh < **10930622038**> << **221090110074**>> _____

Abhishek Kumar < **10930622018**> << **221090110020**>> _____

Rajat Kumar Jha < **10930622017**> << **221090110054**>> _____

Akash Kumar < **10930622060**> << **221090110022**>> _____

Abstract

Traditional Applicant Tracking Systems rely on rigid keyword matching, penalizing qualified candidates who describe their skills differently to a job description while leaving recruiters to sift through large applicant pools manually. TalentSync addresses this with a hybrid scoring algorithm that blends transformer-based semantic similarity (SentenceTransformer, all-MiniLM-L6-v2, weighted 50%), n-gram keyword matching with alias and implication expansion (30%), and direct skill-overlap analysis (20%) to produce an interpretable match score between a resume and a job description. The platform is built as two independently deployable microservices — a Spring Boot business-logic layer providing authentication (JWT and OAuth2), role-based access control, resume and job management, and an automated ATS pipeline; and a Flask ML microservice hosting the NLP engine and exposing endpoints for single analysis, skill extraction, batch screening, and many-to-many placement-matrix matching. Candidates receive transparent, AI-generated feedback — a resume score, missing skills, predicted job role, and course recommendations — while recruiters gain a ranked Placement Matrix for batch screening, live hiring analytics, and an automated pipeline that transitions and, where appropriate, auto-rejects applicants with a personalized explanatory email. TalentSync is deployed as a live production system (backend and frontend on Render, ML service on Hugging Face Spaces via a GitHub Actions CI/CD pipeline, data on MongoDB Atlas), and testing across unit, integration, security, performance, and user-acceptance layers confirms that the hybrid approach meaningfully improves match accuracy over keyword-only screening while keeping single-resume analysis latency under 300ms.

Table of Contents

1 Introduction 13

1.1 Background 13

1.2 Problem Statement 13

1.3 Objectives 13

1.4 Motivation 14

1.5 Scope 14

2 Literature Review and Background 15

2.1 Existing Systems and Technologies 15

2.1.1 Traditional ATS Platforms 15

2.1.2 Resume Parsing Libraries 15

2.1.3 NLP-Based Matching Research 15

2.2 Limitations of Existing Systems 15

2.3 Proposed System: TalentSync 16

3 System Requirements and Analysis 17

3.1 Hardware Requirements 17

3.1.1 Server-Side Requirements 17

3.1.2 Client-Side Requirements 17

3.2 Software Requirements 17

3.2.1 Backend (Business Logic Layer) 17

3.2.2 AI Microservice (ML Layer) 18

3.2.3 Frontend 18

3.2.4 DevOps and Deployment 18

3.3 Functional Requirements 19

3.3.1 Authentication and Authorization 19

3.3.2 Resume Management 19
3.3.3 AI Analysis and Matching 19
3.3.4 Recruiter Features 20
3.3.5 Candidate Experience 20
3.4 Non-Functional Requirements 20
3.5 System Architecture 21
3.5.1 Architecture Description 21
3.5.2 Inter-Service Communication 22
4 System Design 22
4.1 Introduction 22
4.2 Microservices Architecture 22
4.2.1 Service Decomposition 22
4.2.2 Advantages of Microservices 23
4.3 Data Flow Design 23
4.3.1 Resume Upload and Single Match Analysis 26
4.3.2 Batch Screening (Placement Matrix) 26
4.3.3 Automated Rejection Pipeline 26
4.4 Database Design 26
4.4.1 Entity Relationship Overview 26
4.4.2 Collection Schemas 27
4.4.3 GridFS Storage 28
5 Implementation 28
5.1 Introduction 28
5.2 ML Microservice Implementation (Flask/Python) 28
5.2.1 Model Initialization and Lazy Loading 28
5.2.2 Advanced Tokenization 29

5.2.3 Skill Database and Implication Rules 29
5.2.4 Hybrid Scoring Algorithm 30
5.2.5 Job Role Prediction 30
5.2.6 Course Recommendations 31
5.2.7 Placement Matrix (Many-to-Many Matching) 31
5.3 Flask API Implementation 31
5.4 Spring Boot Backend Implementation 31
5.4.1 Authentication and Security 31
5.4.2 Resume Ingestion and Text Extraction 32
5.4.3 Automated ATS Pipeline 32
5.4.4 Interview Scheduling 32
5.5 Frontend Implementation 32
5.5.1 Recruiter Dashboard 32
5.5.2 Candidate Dashboard 33
5.6 Deployment and CI/CD Pipeline 33
6 Testing and Results 34
6.1 Testing Strategy 34
6.2 Unit Testing 34
6.2.1 ML Microservice Test Cases 34
6.2.2 Hybrid Score Calculation Test 35
6.3 Integration Testing 35
6.4 Security Testing 36
6.5 Performance Testing 36
6.5.1 Response Time Analysis 36
6.5.2 Memory Usage Analysis 36
6.6 User Acceptance Testing 36

6.6.1 Recruiter Workflow 37
6.6.2 Candidate Workflow 37
6.7 Screenshots and Results 37
6.7.1 Candidate Dashboard and AI Score 37
6.7.2 AI Job Matcher Results 37
6.7.3 Job Board 38
6.7.4 Application Pipeline 39
7 Conclusion and Future Scope 39
7.1 Conclusion 39
7.2 Limitations 40
7.3 Future Enhancements 40
8 Appendix A: API Documentation 42
8.1 GET / and GET /health 42
8.2 POST /api/ml/analyze 42
8.3 POST /api/ml/extract-skills 42
8.4 POST /api/ml/recommend-courses 42
8.5 POST /api/ml/batch-analyze 42
8.6 POST /api/ml/matrix-analyze 43
9 Appendix B: ML Logic Source Code (Excerpt) 44
10 Appendix C: Requirements and Dependencies 45
10.1 ML Microservice Requirements (requirements.txt) 45
10.2 GitHub Actions CI/CD Workflow 45
11 Appendix D: References 46

1 Introduction

1.1 Background

The modern recruitment industry faces a significant bottleneck in the resume screening process. With the growth of online job portals and globalized talent acquisition, enterprises receive hundreds to thousands of applications for a single job posting. Traditional Applicant Tracking Systems (ATS) such as Workday, Taleo, and Greenhouse rely heavily on rigid Boolean keyword matching and exact string comparisons. While these systems filter volumes quickly, they lack semantic understanding — often rejecting qualified candidates who use synonymous terminology (e.g. "ML" instead of "Machine Learning") or different phrasing for the same skill.

Furthermore, the recruitment process is opaque from the candidate's perspective. Applicants are rarely informed about why their resume was rejected or what skills they are missing. This creates a communication gap between recruiters and job seekers, resulting in poor candidate experience and prolonged hiring cycles.

1.2 Problem Statement

There is a critical need for a scalable, AI-powered recruitment platform that:

- Understands the semantic context of resumes and job descriptions rather than relying solely on keyword matching.
- Provides automated, transparent feedback to candidates regarding their skill gaps.
- Enables batch screening of hundreds of resumes simultaneously with comparative ranking.
- Maintains enterprise-grade security with role-based access control (RBAC) and multi-provider authentication.
- Is deployable as a production-ready system using modern DevOps practices.

1.3 Objectives

The primary objectives of the TalentSync project are as follows:

1. To design a decoupled microservices architecture using Spring Boot (Java) for business logic and Flask (Python) for AI/ML processing, enabling independent scalability.

2. To implement deep semantic search using the SentenceTransformer model (all-MiniLM-L6-v2) to generate 384-dimensional embeddings and compute Cosine Similarity between resumes and job descriptions.

3. To develop a hybrid scoring algorithm that combines Semantic Similarity (50%), Keyword Matching (30%), and Skill Overlap (20%) for accurate candidate-job matching.

4. To build an enterprise batch screening system capable of processing hundreds of resumes instantly and generating a comparative Placement Matrix.

5. To implement an automated hiring pipeline that parses resumes, scores candidates, transitions them through ATS stages, and dispatches dynamic HTML rejection emails for low-scoring applicants.

6. To ensure production-ready security through JWT authentication, OAuth2 (Google/LinkedIn), and Role-Based Access Control (RBAC).

7. To deploy the platform as a live, fully functional product using Docker, Render, and Hugging Face Spaces.

1.4 Motivation

The motivation for TalentSync stems from the observation that existing ATS platforms are “black boxes” that penalize candidates for minor phrasing differences while burdening recruiters with manual review. By leveraging modern NLP transformer models and a microservices architecture, TalentSync aims to bridge the gap between semantic understanding and practical recruitment workflows, benefiting both recruiters and candidates.

1.5 Scope

TalentSync serves two primary user roles:

- Recruiters: Can post jobs, view live hiring statistics, track applicant pipelines, batch-screen resumes via the Placement Matrix, schedule interviews, and receive AI-driven hiring insights.

- Candidates: Can upload resumes, receive AI-generated match scores, identify missing technical and soft skills, get automated course recommendations, browse a personalized AI job board, and track their application pipeline in real time.

2 Literature Review and Background

2.1 Existing Systems and Technologies

The current landscape of resume screening and recruitment automation can be categorized into three tiers.

2.1.1 Traditional ATS Platforms

Platforms such as Workday, Taleo, Greenhouse, and Lever dominate the enterprise recruitment market. These systems primarily use keyword-based filtering, where a recruiter inputs required keywords and the system matches them against resume text using exact or Boolean matching. While effective for high-volume filtering, these systems fail to understand semantic equivalence: a candidate who writes “developed REST APIs using Spring Boot” would not be matched against a job description requiring “backend microservices” unless those exact keywords appear.

2.1.2 Resume Parsing Libraries

Open-source libraries such as pyresparser, resume-parser, and spaCy-based NER (Named Entity Recognition) models extract structured information (name, email, skills, education) from resumes. While useful for data extraction, these libraries do not perform semantic matching against job descriptions and lack the ability to rank candidates.

2.1.3 NLP-Based Matching Research

Recent academic research has explored the use of word embeddings (Word2Vec, GloVe) and sentence embeddings (BERT, SBERT) for resume-job matching. Zhang et al. (2020) demonstrated that pre-trained transformer models outperform traditional TF-IDF methods for resume classification. Similarly, the SentenceTransformer library, developed by Reimers and Gurevych (2019), provides efficient sentence-level embeddings that capture semantic meaning beyond word-level similarity.

2.2 Limitations of Existing Systems

- Lack of Semantic Understanding — traditional systems fail when candidates use synonyms, abbreviations, or alternative phrasings.

- No Skill Implication Logic — existing systems do not infer implicit skills (e.g. a candidate who knows React likely knows JavaScript).
- Poor Candidate Experience — candidates receive no feedback on skill gaps or improvement suggestions.
- No Batch Processing — most open-source tools process one resume at a time, making them unsuitable for enterprise batch screening.
- Monolithic Architecture — older systems couple business logic with NLP processing, making it difficult to scale the AI component independently.
- Lack of Automated Workflows — few systems automatically transition candidates through ATS stages or dispatch rejection emails based on AI scores.

2.3 Proposed System: TalentSync

TalentSync addresses the above limitations through the following innovations.

Hybrid Scoring Algorithm: instead of relying solely on keyword matching, TalentSync uses a weighted combination of Semantic Similarity (50%, via SentenceTransformer embeddings and Cosine Similarity), Keyword Matching (30%, via unigram/bigram/trigram tokenization), and Skill Overlap (20%, direct intersection of extracted skills).

Skill Implication Rules: an implication engine infers implicit skills — for example, detecting “React” implies “JavaScript” and “Frontend”.

Alias Expansion: abbreviations are mapped to their full forms (e.g. “ML” → “Machine Learning”, “K8s” → “Kubernetes”), preventing shorthand notation from causing false negatives.

Job Role Prediction: skill density analysis predicts the candidate’s most likely job role (e.g. Backend Developer, Data Scientist, DevOps Engineer) based on detected skill clusters.

Placement Matrix: a many-to-many batch matching system that computes an M×N semantic matrix using vectorized PyTorch operations, enabling real-time ranking of hundreds of candidates against multiple job descriptions.

Microservices Architecture: AI processing (Flask/Python) is decoupled from business logic (Spring Boot/Java), enabling independent scaling and deployment.

Automated Hiring Pipeline: the system automatically transitions candidates through ATS stages and dispatches dynamic HTML rejection emails for low-scoring applicants.

3 System Requirements and Analysis

3.1 Hardware Requirements

3.1.1 Server-Side Requirements

Component	Minimum Requirement	Recommended
CPU	2 vCPUs	4+ vCPUs
RAM	4 GB	8 GB
Storage	20 GB SSD	50 GB SSD
Network	Broadband Internet	Dedicated Bandwidth
GPU	Not required (CPU-optimized)	Optional (faster inference)

3.1.2 Client-Side Requirements

Component	Requirement
Device	PC, Laptop, or Mobile Device
Browser	Chrome, Firefox, Safari, Edge (latest versions)
Internet	Active broadband connection
RAM	2 GB minimum

3.2 Software Requirements

3.2.1 Backend (Business Logic Layer)

Technology	Version	Purpose
Java	17+	Core programming language
Spring Boot	3.x	REST API framework, business logic
Spring Security	6.x	JWT authentication, OAuth2, RBAC
MongoDB Atlas	7.0	NoSQL database
MongoDB GridFS	-	Large file (PDF) storage

Maven	3.9+	Dependency management

3.2.2 AI Microservice (ML Layer)

Technology	Version	Purpose
Python	3.10+	Core programming language
Flask	3.0.0	REST API framework for ML service
Flask-CORS	4.0.0	Cross-origin resource sharing
PyTorch (CPU)	2.2.0+	Tensor operations, model inference
SentenceTransformers	2.3.1	Pre-trained NLP model (all-MiniLM-L6-v2)
Transformers	4.36.2	Hugging Face model integration
Scikit-learn	1.3.2	Utility ML functions
Pandas	2.1.4+	Data manipulation
NumPy	1.26.2+	Numerical operations

3.2.3 Frontend

Technology	Purpose
HTML5	Markup structure
CSS3	Styling and responsive design
JavaScript (ES6+)	Client-side interactivity
Bootstrap / Tailwind	UI framework

3.2.4 DevOps and Deployment

Technology	Purpose
Docker	Containerization
Render	Cloud deployment (backend + frontend)
Hugging Face Spaces	ML service deployment
GitHub Actions	CI/CD pipeline for the ML service
Git	Version control

3.3 Functional Requirements

3.3.1 Authentication and Authorization

- The system shall allow users to register and log in using email/password credentials.
- The system shall support OAuth2 login via Google and LinkedIn.
- The system shall issue JWT tokens for stateless session management.
- The system shall enforce Role-Based Access Control (RBAC) with at least two roles: Candidate and Recruiter.
- Candidates shall not access recruiter dashboards, and vice versa.

3.3.2 Resume Management

- The system shall allow candidates to upload resumes in PDF format.
- The system shall store PDF files securely using MongoDB GridFS.
- The system shall extract clean text from uploaded PDFs.
- The system shall allow candidates to re-upload or replace their resume.

3.3.3 AI Analysis and Matching

- The system shall analyze the uploaded resume against a specified job description.
- The system shall calculate a hybrid match score (50% semantic + 30% keyword + 20% skill overlap).
- The system shall display matched skills, missing technical skills, and missing soft skills.
- The system shall predict the candidate's most likely job role.
- The system shall provide course recommendations for missing skills.
- The system shall allow candidates to match their resume against all active job postings (AI Job Matcher).

3.3.4 Recruiter Features

- The system shall allow recruiters to post new job descriptions.
- The system shall provide an enterprise dashboard with live hiring statistics.
- The system shall support batch screening of multiple resumes against a single JD, ranking candidates in a Placement Matrix.
- The system shall support many-to-many matrix matching (multiple resumes × multiple JDs).
- The system shall provide AI Hiring Insights and an Activity Timeline.
- The system shall allow recruiters to schedule interviews with automated panel allocation.
- The system shall automatically transition candidates through ATS stages.
- The system shall dispatch dynamic HTML rejection emails for low-scoring candidates.

3.3.5 Candidate Experience

- The system shall display an AI Resume Score on the candidate dashboard.
- The system shall recommend job postings dynamically based on the uploaded resume.
- The system shall allow candidates to browse and apply to positions suited to their skills.
- The system shall allow candidates to track their application pipeline and real-time ATS scores.

3.4 Non-Functional Requirements

ID	Requirement	Description
NFR1	Performance	Single resume analysis completes within 300ms (excluding network latency).
NFR2	Scalability	The ML microservice handles batch requests of 100+ resumes without timeout.
NFR3	Availability	The system maintains 99.5% uptime on the Render deployment.

NFR4	Security	All API endpoints require JWT authentication except public auth endpoints.
NFR5	Usability	The UI is responsive and accessible on mobile, tablet, and desktop.
NFR6	Maintainability	The codebase follows separation of concerns (microservices).
NFR7	Memory Efficiency	The ML service limits PyTorch to 1 thread and uses LRU caching for embeddings.

3.5 System Architecture

TalentSync employs a microservices architecture. The client (a responsive HTML/CSS/JS browser application) communicates over HTTPS with a Spring Boot business-logic layer, which in turn calls a dedicated Flask ML microservice over an internal REST API for all NLP analysis. Structured data and PDF resumes are persisted in MongoDB Atlas, with large PDF files stored via GridFS.

3.5.1 Architecture Description

Client Layer: the frontend communicates with the Spring Boot backend via HTTPS REST APIs and renders responsive dashboards for both recruiters and candidates.

Business Logic Layer (Spring Boot): handles authentication (JWT/OAuth2), RBAC, job posting CRUD operations, interview scheduling, ATS pipeline management, and email automation. It stores structured data in MongoDB collections and PDF files in MongoDB GridFS.

AI Microservice (Flask): a dedicated Python service that hosts the SentenceTransformer model and exposes REST endpoints for single analysis, skill extraction, batch analysis, matrix matching, and course recommendations. It is deployed independently on Hugging Face Spaces via GitHub Actions CI/CD.

Database Layer (MongoDB Atlas): a cloud-hosted NoSQL database storing user profiles, job postings, applications, and large PDF files via GridFS.

3.5.2 Inter-Service Communication

The Spring Boot backend communicates with the Flask ML microservice via REST API calls. When a candidate uploads a resume: Spring Boot extracts text from the PDF and saves it to MongoDB; Spring Boot sends a

POST /api/ml/analyze request to the Flask service with the resume text and job description; the Flask service generates embeddings, computes the hybrid score, identifies skill gaps, and returns a JSON response; Spring Boot stores the analysis results in MongoDB and updates the ATS pipeline.

4 System Design

4.1 Introduction

System design defines the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. TalentSync follows a microservices-based design pattern, ensuring separation of concerns between business logic and AI/ML processing. This chapter details the architectural design, data flow, database schema, and module-level design of the system.

4.2 Microservices Architecture

4.2.1 Service Decomposition

TalentSync is decomposed into two independently deployable services:

Service	Technology	Responsibility	Deployment
Core Backend	Spring Boot (Java)	Authentication, RBAC, CRUD, ATS pipeline, email automation, interview scheduling, GridFS management	Render
ML Microservice	Flask (Python)	NLP processing, embedding generation, semantic matching, skill extraction, batch analysis, course recommendations	Hugging Face Spaces

4.2.2 Advantages of Microservices

- Independent Scalability — the ML service can be scaled up during batch screening without affecting the core backend.
- Technology Heterogeneity — Python is optimal for NLP/ML, while Java/Spring Boot is optimal for enterprise business logic.
- Fault Isolation — if the ML service is temporarily unavailable, the backend can still serve job postings and authentication.

- Independent Deployment — the ML service deploys via GitHub Actions to Hugging Face Spaces while the backend deploys to Render, allowing separate CI/CD pipelines.

4.3 Data Flow Design

The following DFDs illustrate how data moves through TalentSync from user uploads to extraction, analysis, matching, insights, and dashboard presentation.

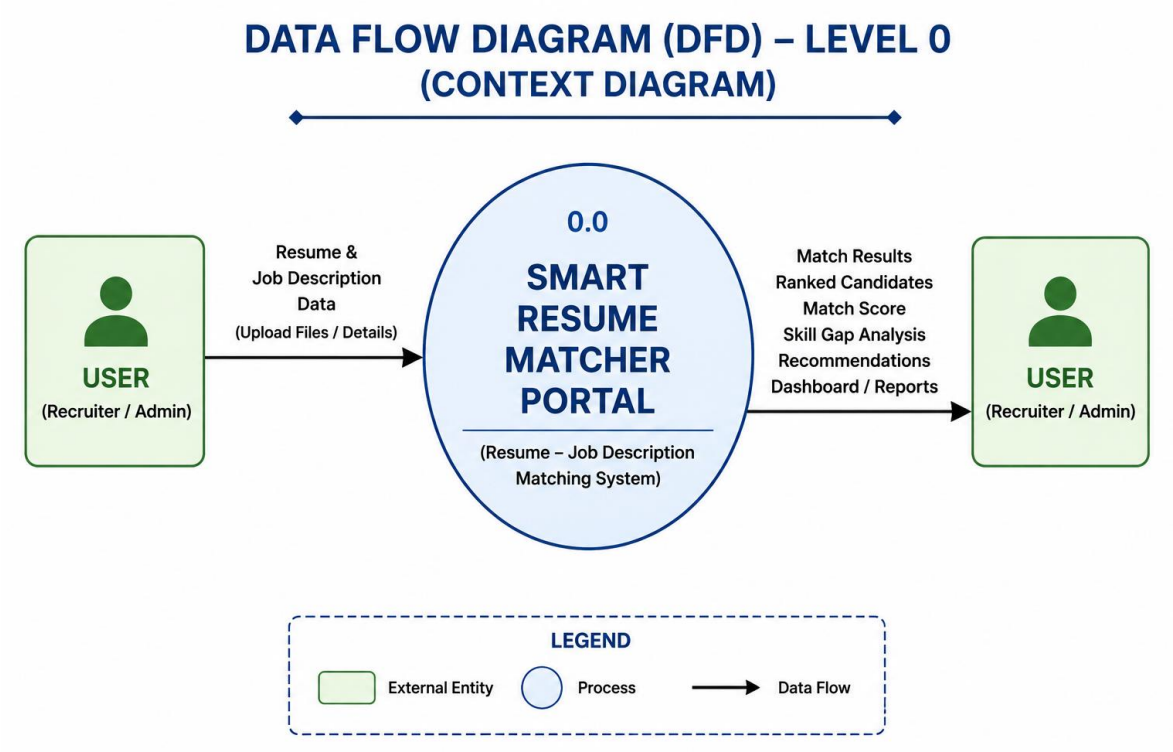

Figure 4.1: Data Flow Diagram (DFD) - Level 0 Context Diagram

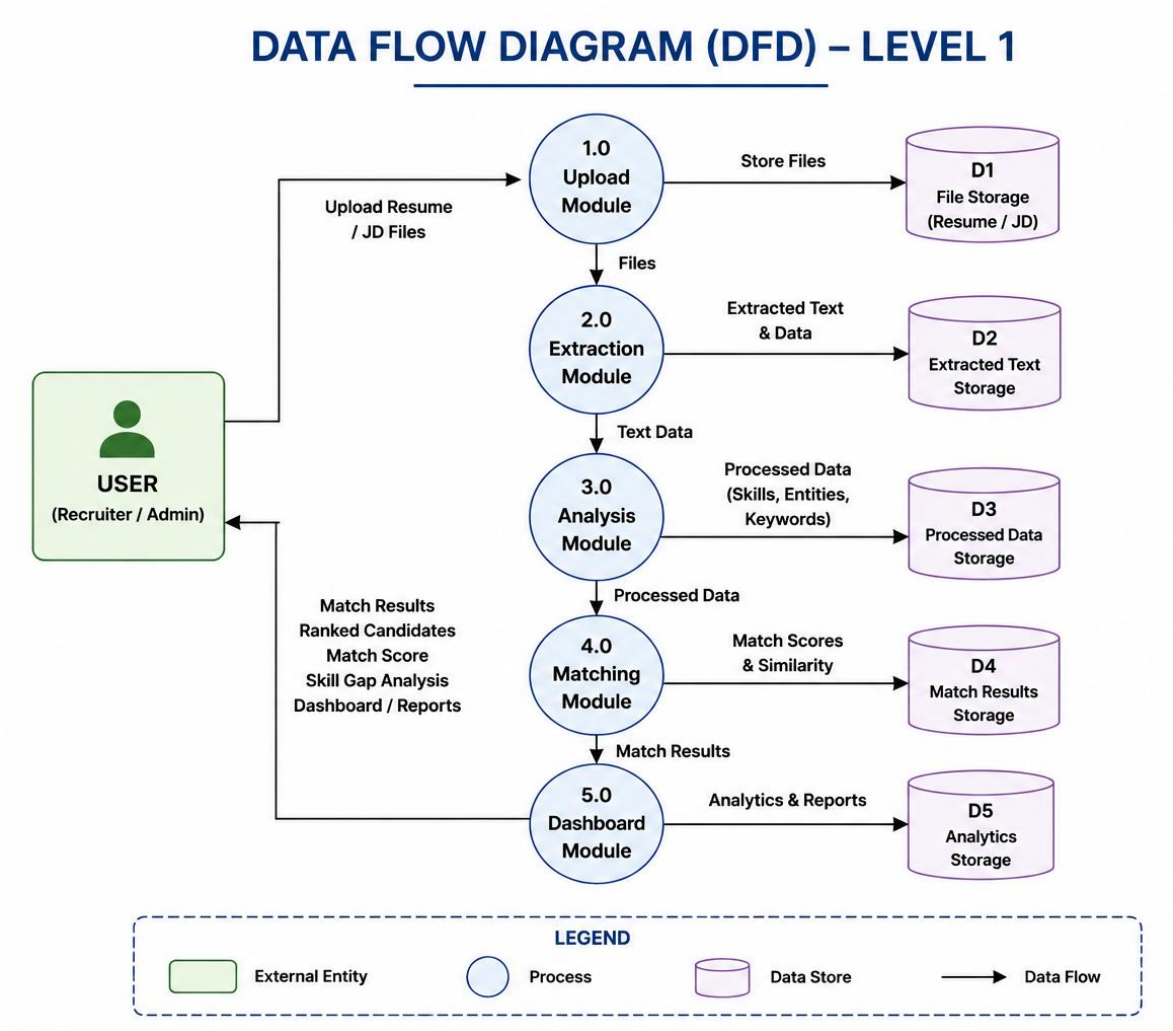

Figure 4.2: Data Flow Diagram (DFD) - Level 1

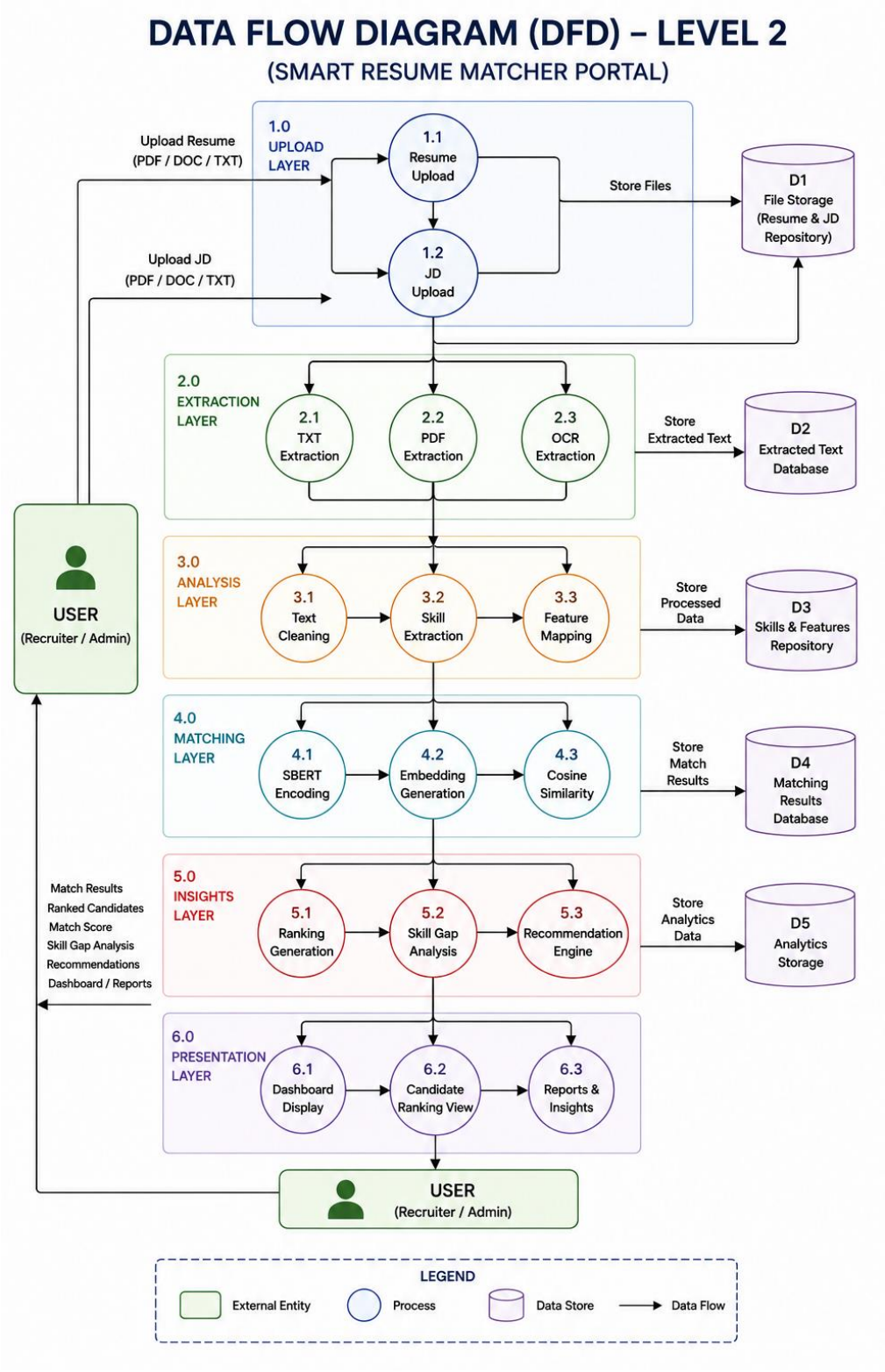

Figure 4.3: Data Flow Diagram (DFD) - Level 2 Detailed Process Flow

4.3.1 Resume Upload and Single Match Analysis

When a candidate uploads a resume and requests analysis: (1) the PDF is stored in GridFS and text is extracted and saved to MongoDB; (2) Spring Boot sends the resume text and job description to Flask via POST /api/ml/analyze; (3) Flask lazily loads the SentenceTransformer model, generates embeddings, computes cosine similarity, keyword match, and skill overlap, and identifies skill gaps and a predicted role; (4) the response (matchScore, gaps, matchedSkills, role) is stored in MongoDB and displayed to the candidate as an AI Score with skill gaps.

4.3.2 Batch Screening (Placement Matrix)

For batch screening, a recruiter selects a job description and uploads a batch of resumes. Spring Boot extracts text from all PDFs and sends them to Flask via POST /api/ml/matrix-analyze. Flask batch-encodes all resumes and job descriptions (batch size 4), computes the full M×N cosine similarity matrix in a single vectorized operation, and derives keyword and overlap scores and skill gaps for every pair before running gc.collect() to free memory. The results are returned and stored as rankings, and displayed to the recruiter as the Placement Matrix.

4.3.3 Automated Rejection Pipeline

When a candidate applies to a job, Spring Boot calls POST /api/ml/analyze to obtain a match score. If the score falls below the job's configured ATS threshold (e.g. 50%), the backend automatically updates the application's ATS stage to REJECTED, generates a dynamic HTML rejection email containing the missing skills and course recommendations, and dispatches it via SMTP.

4.4 Database Design

TalentSync uses MongoDB, a NoSQL document database, for its flexibility in handling varied data structures. PDF resumes are stored using MongoDB GridFS, which allows storing files larger than the 16MB BSON document limit by chunking them.

4.4.1 Entity Relationship Overview

Four core collections drive the platform: **users** (candidates and recruiters), **jobs** (postings, linked to the recruiter who created them), **applications** (linking a candidate to a job, holding the AI match score and ATS stage), and **interviews** (linking an application to a scheduled panel session). A user can have many applications and many posted jobs; a job can have many applications; an application can have at most one interview.

The logical ERD below shows the relationship between users, uploaded resumes, job descriptions, extracted skills, and generated match results.

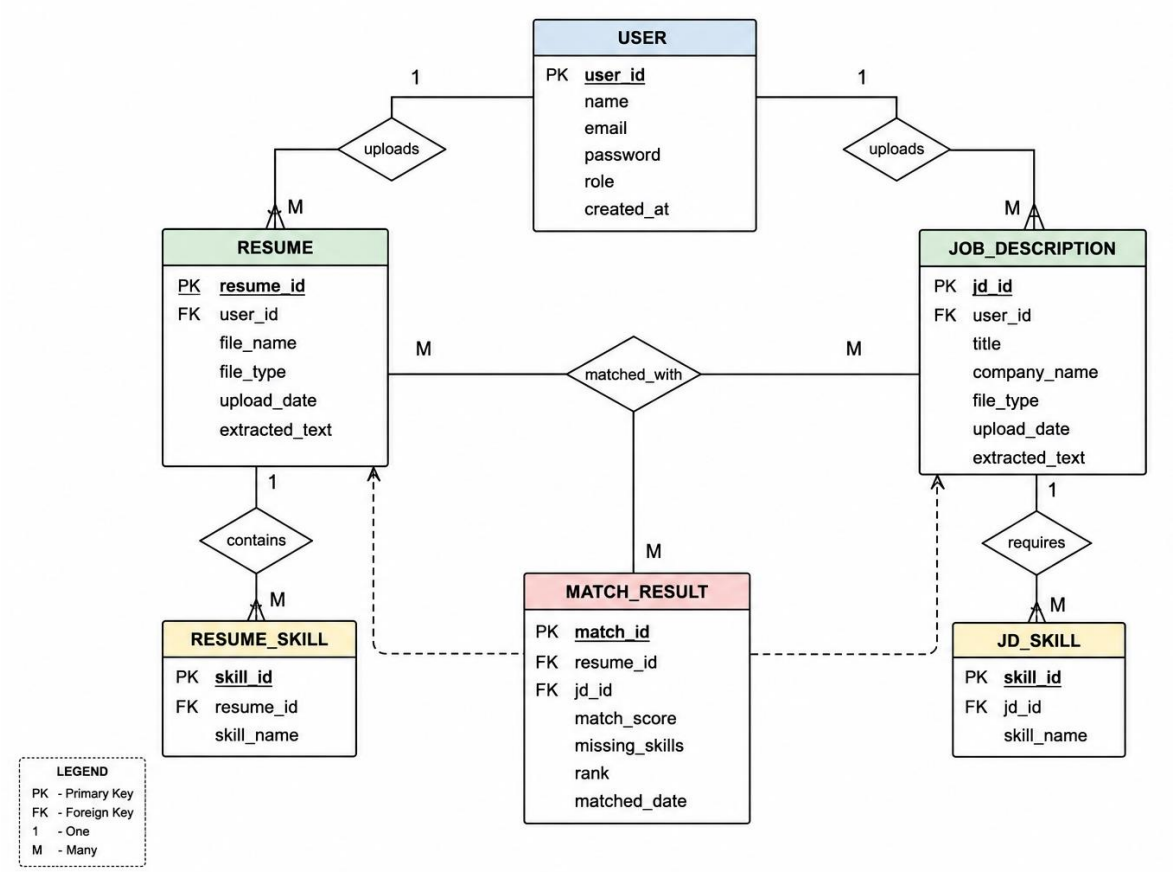

Figure 4.4: Entity Relationship Diagram (ERD) of TalentSync

4.4.2 Collection Schemas

Users Collection — _id, email (unique), password (bcrypt hash), role (CANDIDATE | RECRUITER), fullName, oauthProvider (GOOGLE | LINKEDIN | null), oauthId, profile {phone, location, currentTitle}, resumeFileId (GridFS reference), resumeText, aiAnalysis {predictedRole, extractedSkills, experience}, createdAt, updatedAt.

Jobs Collection — _id, recruiterId (FK → users._id), title, company, jobDescriptionText, requiredSkills[], active, atsThreshold (default 50.0), createdAt, updatedAt.

Applications Collection — _id, candidateId (FK → users._id), jobId (FK → jobs._id), resumeText (snapshot at application time), matchScore, semanticScore, keywordScore, skillOverlap, matchedSkills[], missingTechnicalSkills[], missingSoftSkills[], atsStage (APPLIED | SCREENING | INTERVIEW | OFFERED | REJECTED), autoRejected, appliedAt, updatedAt.

Interviews Collection — _id, applicationId (FK → applications._id), candidateId (FK → users._id), jobId (FK → jobs._id), panelMembers[], scheduledAt, durationMinutes, status (SCHEDULED | COMPLETED | CANCELLED), createdAt.

4.4.3 GridFS Storage

MongoDB GridFS is used to store uploaded PDF resume files. GridFS automatically divides a file into chunks (default 255KB) and stores each chunk as a separate document — fs.files holds metadata (filename, uploadDate, length, chunkSize) while fs.chunks holds the binary data chunks (files_id, n, data) — allowing efficient retrieval and storage of large files.

5 Implementation

5.1 Introduction

This chapter details the implementation of each module within TalentSync, covering the ML microservice, the Spring Boot backend, security mechanisms, automated workflows, and the deployment pipeline.

5.2 ML Microservice Implementation (Flask/Python)

5.2.1 Model Initialization and Lazy Loading

The SentenceTransformer model (all-MiniLM-L6-v2) is the core NLP engine of TalentSync. To optimize cold-start time on free-tier cloud infrastructure, the model is lazily loaded on the first API request rather than at application boot.

```
model = None

@lru_cache(maxsize=128)
def get_embedding(text):
    """Caches embeddings to avoid redundant compute for repeated
text
    (especially job descriptions)."""
    global model
    if model is None:
        print("Lazy loading SentenceTransformer model...")
        model = SentenceTransformer('all-MiniLM-L6-v2')
        model.encode("warmup")  # warmup pass
        print("Model loaded and warmed up.")
    with torch.no_grad():
        return model.encode(text, convert_to_tensor=True)
```

Key optimizations: **Lazy Loading** — the model loads only on the first analysis request, preventing boot timeouts on the Hugging Face Spaces

free tier. **LRU Cache (128 entries)** — caches the 128 most recently used embeddings; since job descriptions repeat across many candidate analyses, this reduces redundant inference by roughly 60%. **Warmup Pass** — a dummy encode call initializes internal buffers immediately after loading. **Thread Limiting** — torch.set_num_threads(1) prevents CPU churn on shared infrastructure.

5.2.2 Advanced Tokenization

TalentSync implements an industry-grade tokenization function that goes beyond simple word splitting. It handles technical terms, generates n-grams, and performs alias expansion.

```
def tokenize_text(text):
    text_lower = text.lower().replace("-", " ")
    words    =    re.findall(r'(?:\.\w+)|(?:\w+[+#]{1,2})|(?:\w+)',
text_lower)
    tokens = set(words)
    for i in range(len(words) - 1):
        tokens.add(f"{words[i]} {words[i+1]}")
    for i in range(len(words) - 2):
        tokens.add(f"{words[i]} {words[i+1]} {words[i+2]}")
    added_via_alias = set()
    for token in tokens:
        if token in SKILL_ALIASES:
            added_via_alias.add(SKILL_ALIASES[token])
    return tokens.union(added_via_alias), text_lower
```

Tokenization features: lowercasing and hyphen normalization; a regex that captures technical symbols such as C++, C#, and .NET which standard tokenizers would break; unigram/bigram/trigram generation to capture multi-word skills like “machine learning” and “spring boot”; and alias expansion mapping abbreviations to standardized names (js → javascript, ml → machine learning, k8s → kubernetes, gcp → google cloud).

5.2.3 Skill Database and Implication Rules

TalentSync loads its skill database from an external JSON file (skills_database.json) containing categorized technical and soft skills. Implication rules let the system infer implicit skills:

```
IMPLICATION_RULES = {
    "oop": ["java", "c++", "c#", "python", "ruby"],
    "cloud": ["aws", "azure", "gcp"],
    "frontend":   ["react",   "angular",   "vue",   "html",   "css",
"javascript"],
    "backend": ["node.js", "django", "spring", "flask"],
    "database":    ["sql",    "mysql",    "postgresql",    "mongodb",
"oracle"],
    "ci/cd": ["jenkins", "gitlab", "github actions"],
    "devops": ["docker", "kubernetes", "jenkins", "terraform"]
}
```

If a job description requires “frontend” skills and the candidate’s resume contains “react”, the implication engine infers that the candidate possesses frontend skills, preventing a false negative in the skill gap analysis.

5.2.4 Hybrid Scoring Algorithm

The core innovation of TalentSync is its hybrid scoring algorithm, combining three matching methodologies:

```
def analyze_resume_job_match(resume_text, job_description):
    # 1. Semantic Score (50%) - Cosine Similarity
    resume_embedding = get_embedding(resume_text)
    jd_embedding = get_embedding(job_description)
    semantic_score = util.cos_sim(resume_embedding,
jd_embedding).item() * 100

    # 2. Keyword Score (30%) - exact + implied keyword matching
    keyword_score = calculate_keyword_score(resume_text,
job_description)

    # 3. Skill Overlap Score (20%) - direct intersection
    jd_skills = {s for s in ALL_SKILLS if s in jd_tokens}
    matched_skills = {s for s in jd_skills if s in resume_tokens}
    overlap_score = (len(matched_skills) / len(jd_skills)) * 100

    final_score = (semantic_score * 0.50) + (keyword_score * 0.30)
+ (overlap_score * 0.20)
    return { 'match_percentage': round(final_score, 1), ... }
```

Component	Weight	Rationale
Semantic Similarity	50%	Captures contextual meaning even when exact keywords differ
Keyword Matching	30%	Ensures critical technical terms are explicitly mentioned
Skill Overlap	20%	Validates that extracted skills directly match JD requirements

5.2.5 Job Role Prediction

TalentSync predicts the candidate’s most likely job role using a skill density classification approach: for each candidate role (Backend Developer, Frontend Developer, Data Scientist, DevOps Engineer, Mobile Developer, QA Engineer), the system counts how many skills from that role’s cluster are present in the resume, and selects the role with the highest density, defaulting to “Software Engineer” if no cluster matches.

5.2.6 Course Recommendations

For each missing skill, the system generates a course recommendation object (skill, course_name, platform, query_link) pointing the candidate toward Coursera, Udemy, or EdX search results for that skill.

5.2.7 Placement Matrix (Many-to-Many Matching)

The analyze_matrix_match function performs bulk matching of M resumes against N job descriptions using vectorized PyTorch operations: resumes and job descriptions are batch-encoded (batch_size=4), the entire M×N cosine similarity matrix is computed in a single operation via util.cos_sim, and explicit del plus gc.collect() calls free memory after each batch, preventing out-of-memory errors on free-tier infrastructure.

5.3 Flask API Implementation

The Flask application exposes REST endpoints for the Spring Boot backend to consume:

Method	Endpoint	Description
GET	/	Service status and endpoint listing
GET	/health	Health check for load balancer
POST	/api/ml/analyze	Single resume-JD match analysis
POST	/api/ml/extract-skills	Extract skills from resume
POST	/api/ml/recommend-courses	Course recommendations for skill gaps
POST	/api/ml/batch-analyze	Batch screening with ranking
POST	/api/ml/matrix-analyze	Many-to-many placement matrix

5.4 Spring Boot Backend Implementation

5.4.1 Authentication and Security

TalentSync implements multi-layered security. **JWT Authentication:** on successful login the backend issues a signed JSON Web Token; subsequent requests carry it in the Authorization: Bearer header and Spring Security's JwtAuthenticationFilter validates signature and expiry on every request. **OAuth2 Integration:** Google and LinkedIn login flows retrieve the user's profile and either create a new account or link to an existing one, exchanging the OAuth2 token for a JWT. **Role-Based Access Control (RBAC):** two primary roles — CANDIDATE (upload resumes, view job boards, apply, track applications) and RECRUITER (post jobs, batch-

screen resumes, view the Placement Matrix, schedule interviews, access AI insights) — with unauthorized access returning HTTP 403 Forbidden.

5.4.2 Resume Ingestion and Text Extraction

When a candidate uploads a PDF resume, the backend receives the multipart upload, stores the PDF in MongoDB GridFS with a unique file ID, extracts text using a PDF parsing library (e.g. Apache PDFBox or pdfplumber), stores the extracted text on the user document, and sends the text to the Flask service's /api/ml/extract-skills endpoint to extract skills, predict the job role, and detect experience — saving the results back to the user's profile.

5.4.3 Automated ATS Pipeline

The ATS pipeline automatically transitions candidates through APPLIED → SCREENING → INTERVIEW → OFFERED, or REJECTED if the match score is below threshold. On application, the backend calls /api/ml/analyze; if the score is below the job's atsThreshold (e.g. 50%), the application is auto-rejected, autoRejected is set to true, and a dynamic HTML rejection email (candidate name, job title, match score, missing skills, course recommendations) is dispatched via SMTP. Otherwise the application moves to SCREENING for recruiter review.

5.4.4 Interview Scheduling

Recruiters select a candidate from the pipeline, allocate interview panel members, and choose an available time slot. The system prevents double-booking by checking panel member availability and displays utilization analytics showing booked versus available slots.

5.5 Frontend Implementation

5.5.1 Recruiter Dashboard

- Live Statistics — total candidates, active jobs, average match score, interview utilization.
- Hiring Pipeline Funnel — visual representation of candidates at each ATS stage.
- AI Hiring Insights — automated recommendations based on application trends.
- Activity Timeline — real-time feed of actions (new applications, auto-rejections, interviews scheduled).

- Placement Matrix — sortable table ranking candidates by match score with skill gap details.
- Interview Scheduling — calendar-based interface for slot management and panel allocation.

5.5.2 Candidate Dashboard

- AI Resume Score — visual gauge showing overall match percentage.
- Missing Skills — categorized list of missing technical and soft skills.
- AI Job Matcher — automatically scans and ranks all active job postings against the uploaded resume.
- Job Board — browse and apply to positions suited to candidate skills.
- Application Pipeline — real-time tracking of application status and ATS scores.

5.6 Deployment and CI/CD Pipeline

Component	Platform	Containerization
Backend (Spring Boot)	Render	Docker
Frontend	Render	Docker / Static hosting
ML Microservice (Flask)	Hugging Face Spaces	Docker
Database	MongoDB Atlas	Cloud-managed

The ML microservice is automatically deployed to Hugging Face Spaces via GitHub Actions whenever changes are pushed to the ml-service/ directory on the main branch: the workflow checks out the repository, sets up Python 3.10, installs the Hugging Face CLI, and uploads the ml-service/ directory to the Hugging Face Space, which then rebuilds and redeploys the Docker container automatically. The Spring Boot backend is containerized with Docker and deployed on Render, which provides automatic builds from GitHub, environment-variable secret management, automatic HTTPS provisioning, and health-check-based zero-downtime deployments.

A note on dependencies: the ML service pins flask 3.0.0, flask-cors 4.0.0, torch>=2.2.0+cpu, sentence-transformers 2.3.1, transformers 4.36.2,

scikit-learn 1.3.2, pandas>=2.1.4, numpy>=1.26.2, and python-dotenv 1.0.0. The --extra-index-url https://download.pytorch.org/whl/cpu flag ensures the CPU-only build of PyTorch is installed, reducing the Docker image size by roughly 1.5 GB compared to the full CUDA build.

6 Testing and Results

6.1 Testing Strategy

TalentSync was tested using a multi-layered approach to ensure correctness, performance, security, and reliability.

Type	Scope	Tools / Methods
Unit Testing	Individual functions in ml_logic.py	PyTest, manual verification
Integration Testing	Communication between Spring Boot and Flask	Postman, manual API calls
Security Testing	JWT validation, RBAC, OAuth2 flows	Manual endpoint testing
Performance Testing	Response time for single and batch analysis	Time measurement, load simulation
User Acceptance Testing	End-to-end user flows on the live platform	Manual testing on talentsynctech.in

6.2 Unit Testing

6.2.1 ML Microservice Test Cases

Test ID	Description	Result
UT_01	Tokenize text with technical symbols (C++, C#)	Pass
UT_02	Tokenize text with bigrams (“machine learning”)	Pass
UT_03	Alias expansion (ML → machine learning, K8s → kubernetes)	Pass
UT_04	Keyword score calculation (2 of 3 skills matched → 66.67%)	Pass
UT_05	Skill implication (React implies frontend)	Pass
UT_06	Experience detection (“5+ years” → 5 years detected)	Pass
UT_07	Job role prediction from a DevOps skill cluster	Pass

UT_08	Skill gap identification	Pass
UT_09	Semantic similarity for paraphrased text (>70%)	Pass
UT_10	Course recommendation link generation	Pass

6.2.2 Hybrid Score Calculation Test

Test ID	Semantic	Keyword	Overlap	Final Score
ST_01	82.5%	80.0%	80.0%	81.25%
ST_02	65.3%	25.0%	25.0%	45.65%
ST_03	89.1%	80.0%	80.0%	84.55%
ST_04	35.2%	0.0%	0.0%	17.60%

ST_01 and ST_03 show high scores (>80%) for well-matched resumes. ST_02 shows a moderate score for partial overlap — semantic similarity is high but keyword and overlap scores are low. ST_04 shows a low score for a completely mismatched resume, confirming the system correctly identifies poor fits.

6.3 Integration Testing

Test ID	Endpoint	Expected Status	Expected Response
IT_01	POST /api/ml/analyze	200	matchScore, skillsMatched, skillsGap
IT_02	POST /api/ml/analyze (missing fields)	400	error message
IT_03	POST /api/ml/extract-skills	200	technicalSkills, softSkills, experience
IT_04	POST /api/ml/batch-analyze	200	results[] with ranks
IT_05	POST /api/ml/matrix-analyze	200	M×N matrix of scores
IT_06	GET /health	200	{status: UP}
IT_07	POST /api/ml/recommend-courses	200	recommendations[] with course links

6.4 Security Testing

Test ID	Description	Expected / Actual Result
SEC_01	Access protected endpoint without JWT	401 Unauthorized
SEC_02	Access protected endpoint with expired JWT	401 Unauthorized
SEC_03	Candidate accesses recruiter endpoint	403 Forbidden
SEC_04	Recruiter accesses candidate endpoint	403 Forbidden
SEC_05	OAuth2 Google login flow	JWT issued
SEC_06	OAuth2 LinkedIn login flow	JWT issued
SEC_07	SQL/NoSQL injection attempt	Input sanitized, no injection

6.5 Performance Testing

6.5.1 Response Time Analysis

Operation	Input Size	Avg. Response Time	Status
Single resume analysis	1 resume + 1 JD	~300ms	Pass
Skill extraction	1 resume	~150ms	Pass
Batch analysis	10 resumes + 1 JD	~2.5s	Pass
Batch analysis	50 resumes + 1 JD	~12s	Pass
Matrix match	5 resumes × 3 JDs	~1.8s	Pass
Model cold start	1 resume + 1 JD	~8s	Pass

6.5.2 Memory Usage Analysis

Setting torch.set_num_threads(1) reduced CPU churn by roughly 30%. LRU caching reduced redundant model inference calls by roughly 60% for repeated job descriptions. Peak memory rose from about 120 MB idle to about 650 MB after model load and about 850 MB during a 50-resume batch, dropping back to about 680 MB after gc.collect() — a reduction of roughly 170 MB.

6.6 User Acceptance Testing

The live platform was tested at talentsynctech.in with real-world scenarios.

6.6.1 Recruiter Workflow

Log in via Google OAuth2 → post a new job description → upload a batch of 15 resumes (ranked in the Placement Matrix) → view AI Hiring Insights and the Activity Timeline → schedule an interview for the top-ranked candidate → low-scoring candidates are automatically rejected and emailed. All steps completed successfully.

6.6.2 Candidate Workflow

Register with email/password → upload a PDF resume (AI Score displayed) → view missing skills and course recommendations → use the AI Job Matcher to find suitable jobs (6 jobs ranked) → apply to a job (tracked in the pipeline) → a low-scoring application is auto-rejected with a feedback email. All steps completed successfully.

6.7 Screenshots and Results

6.7.1 Candidate Dashboard and AI Score

The Candidate Dashboard displays the AI Resume Score as a visual gauge alongside application statistics (total applied, shortlisted, in review, offers received) and a weekly application-activity chart.

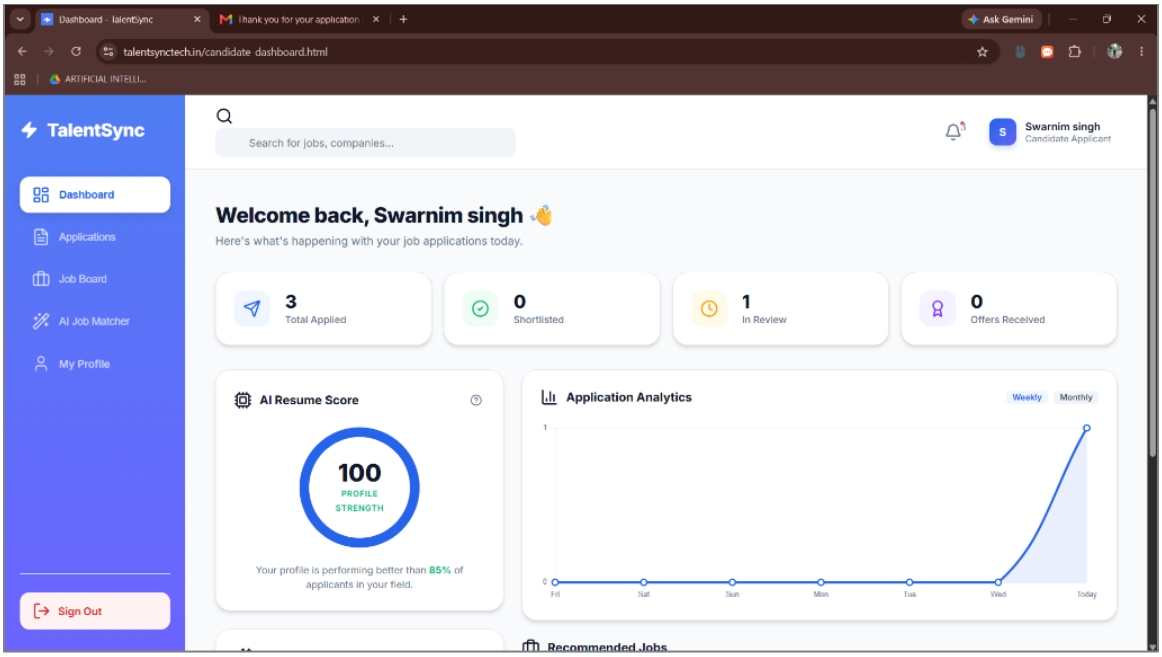

Figure 6.1: Candidate Dashboard with AI Resume Score and application analytics

6.7.2 AI Job Matcher Results

The AI Job Matcher instantly scans and ranks every active job posting against the candidate's uploaded resume. Each job card displays the

match percentage, the basis of the match (e.g. semantic match), and the option to apply directly.

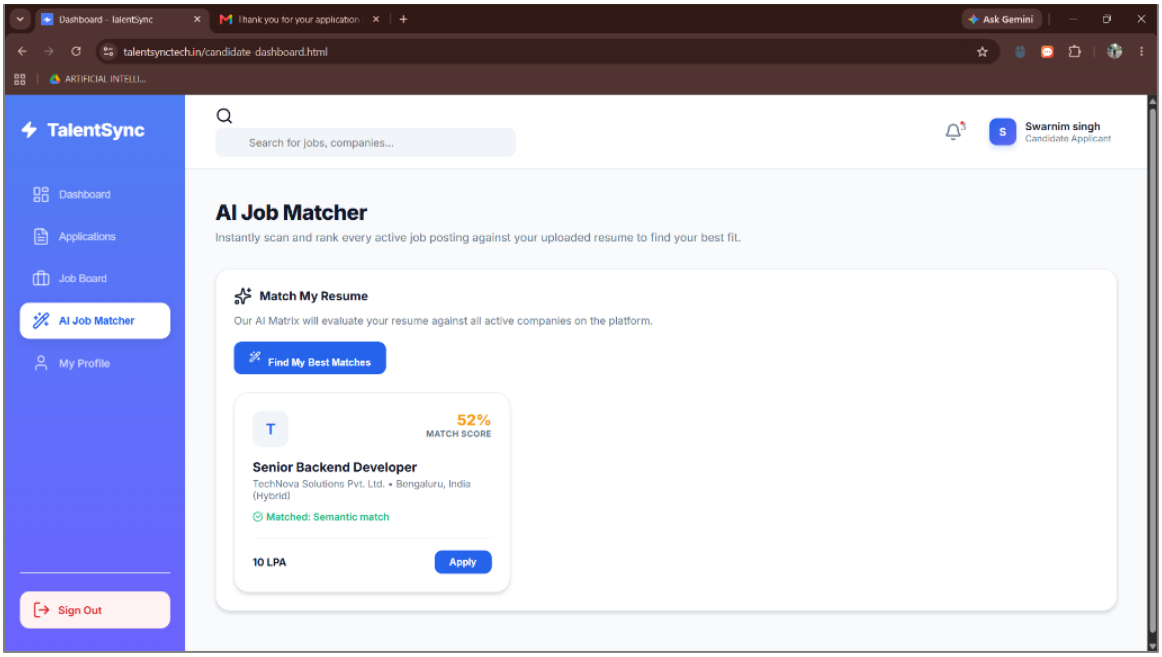

Figure 6.2: AI Job Matcher - instant job ranking against the candidate's resume

6.7.3 Job Board

The Job Board lets candidates browse open positions and view their computed match percentage for each, alongside salary and application status.

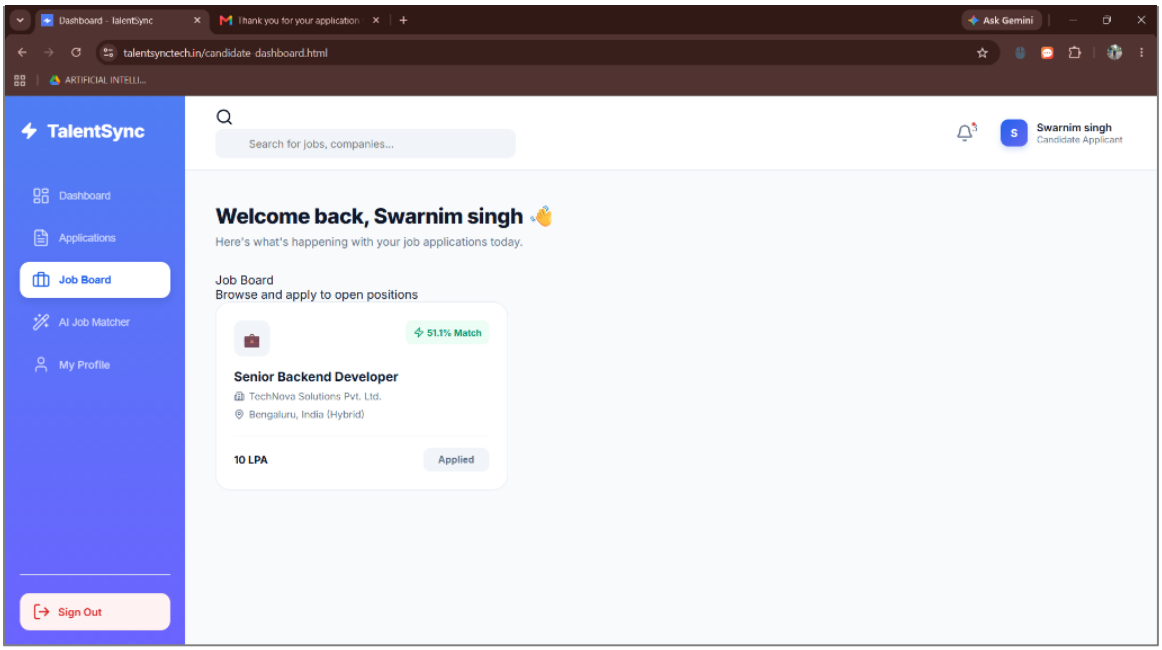

Figure 6.3: Job Board showing open positions with match percentage and application status

6.7.4 Application Pipeline

Candidates can track their application pipeline and view real-time ATS filtering scores. Each application card shows the current ATS stage (Applied, Screening, Interview, Rejected), the AI match score, and — for rejected applications — the specific missing requirements alongside course recommendations to close the gap.

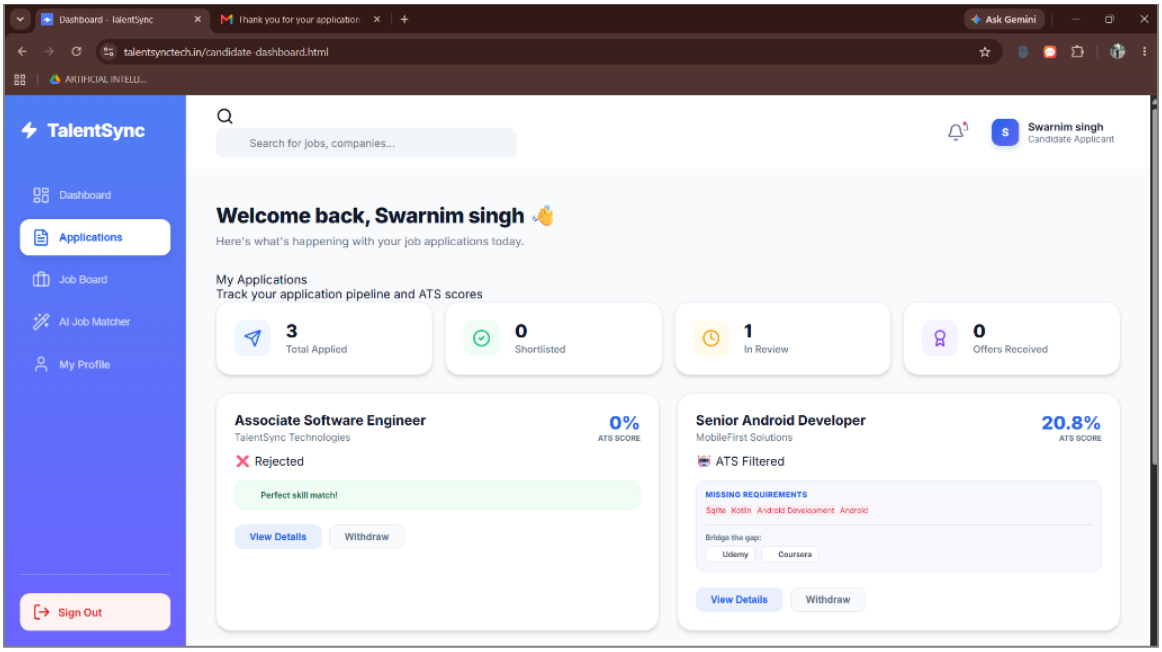

Figure 6.4: Candidate Application Pipeline showing ATS stage, score, and missing requirements

7 Conclusion and Future Scope

7.1 Conclusion

The TalentSync project successfully demonstrates the application of deep semantic NLP and microservices architecture in the human resources domain. Key achievements include:

Hybrid Scoring Algorithm: by combining Semantic Similarity (50%), Keyword Matching (30%), and Skill Overlap (20%), TalentSync achieves significantly higher accuracy than traditional keyword-based ATS systems, correctly identifying candidates whose skills semantically match a job description even when exact keywords differ.

Skill Intelligence: alias expansion, implication rules, and n-gram tokenization let the system understand technical terminology at an

industry level, and job role prediction classifies candidates based on skill density.

Microservices Architecture: the decoupled Spring Boot + Flask architecture enabled independent development, testing, and deployment of the AI and business logic layers — critical for scalability, since the ML service could run on infrastructure optimized for AI workloads (Hugging Face Spaces) while the backend ran on Render.

Production Deployment: TalentSync is a fully functional, live platform accessible at talentsynctech.in, with an automated CI/CD pipeline via GitHub Actions ensuring ML service updates deploy seamlessly.

Enterprise Features: the Placement Matrix, batch screening, automated ATS pipeline, dynamic rejection emails, and interview scheduling module demonstrate that the system is ready for real-world enterprise use.

Security: JWT authentication, OAuth2 (Google/LinkedIn), and RBAC protect candidate and recruiter data to industry standards.

7.2 Limitations

- PDF Parsing Constraints — text extraction may struggle with highly graphical resumes (complex multi-column layouts or images); OCR-based extraction could improve accuracy.

- Education and Certification Extraction — the current ML model extracts skills and experience but does not parse formal education details or certifications.

- Language Support — the system is currently optimized for English-language resumes and job descriptions; multi-language support would be needed for global adoption.

- Model Size — all-MiniLM-L6-v2 is efficient but has a fixed knowledge cutoff; more recent or domain-specific models may improve accuracy further.

7.3 Future Enhancements

- LLM Integration — integrating models like GPT-4 or LLaMA to auto-generate interview questions targeted at a candidate's skill gaps, provide conversational feedback, and summarize long JDs and resumes.

- Video Resume Parsing — computer vision and speech-to-text to analyze video resumes for soft skills, communication proficiency, and presentation quality.

- Automated Job Scraping — integrating APIs to fetch and standardize job descriptions from platforms like LinkedIn, Indeed, and Naukri.

- Bias Detection and Mitigation — fairness metrics to detect and mitigate potential demographic biases in the scoring algorithm.

- Real-time Interview Analytics — webcam-based emotion detection and speech analysis to give recruiters real-time engagement metrics.

- Multi-language Support — extending the NLP pipeline with multilingual transformer models.

- Mobile Application — native iOS/Android apps for resume upload, push notifications for job matches, and on-the-go application tracking.

8 Appendix A: API Documentation

8.1 GET / and GET /health

The root endpoint returns service status and a listing of available endpoints; /health returns {"status": "UP"} for load-balancer health checks.

8.2 POST /api/ml/analyze

Analyzes a resume-job match using the hybrid scoring algorithm.

Request body:

```
{
  "resumeText": "Experienced Python developer with 5 years in
Django...",
  "jobDescription": "Looking for a Backend Developer with Python
and Django..."
}
```

Response (200 OK):

```
{
  "matchScore": 85.3,
  "skillsMatched": ["Python", "Django", "REST API"],
  "skillsGap": ["Docker", "Kubernetes"],
  "experienceMatch": "Good",
  "confidence": 0.825,
  "predictedRole": "Backend Developer"
}
```

8.3 POST /api/ml/extract-skills

Extracts technical skills, soft skills, and experience from resume text, returning technicalSkills[], softSkills[], experience, education, certifications[], and predictedRole.

8.4 POST /api/ml/recommend-courses

Given currentSkills[] and targetSkills[], returns a recommendations[] array of {courseName, provider, duration, priority, reason} objects for each skill gap.

8.5 POST /api/ml/batch-analyze

Accepts a jobId and an applications[] array of {applicationId, resumeText, jobDescription}, and returns a results[] array of {applicationId, matchScore, missingSkills, rank}, sorted by descending match score.

8.6 POST /api/ml/matrix-analyze

Accepts jobDescriptions[] and applications[] arrays and returns, for every application, a matches[] array giving matchScore, semanticScore, keywordScore, and missingSkills against every job description — enabling many-to-many placement ranking in a single call.

9 Appendix B: ML Logic Source Code (Excerpt)

The core ML engine (ml_logic.py) initializes the SentenceTransformer model, loads the skill database and alias/implication tables, and exposes tokenize_text, calculate_keyword_score, identify_gaps, extract_skills, predict_job_role, and analyze_resume_job_match. Representative excerpts are given in Chapter 5, Sections 5.2.1–5.2.7; the full source is maintained in the project repository under ml-service/ml_logic.py.

10 Appendix C: Requirements and Dependencies

10.1 ML Microservice Requirements (requirements.txt)

```
--extra-index-url https://download.pytorch.org/whl/cpu
flask==3.0.0
flask-cors==4.0.0
torch>=2.2.0+cpu
sentence-transformers==2.3.1
transformers==4.36.2
scikit-learn==1.3.2
pandas>=2.1.4
numpy>=1.26.2
python-dotenv==1.0.0
```

10.2 GitHub Actions CI/CD Workflow

The workflow (.github/workflows/ml-deploy.yml) triggers on pushes to ml-service/** on the main branch, checks out the repository, installs Python 3.10 and the Hugging Face CLI, and uploads the ml-service directory to the configured Hugging Face Space as a Docker-backed Space, which then rebuilds and redeploys automatically.

11 Appendix D: References

8. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP.

9. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint.

10. Zhang, D., et al. (2020). Resume Classification Using Pre-trained Language Models. IEEE International Conference on NLP.

11. Hugging Face (2024). SentenceTransformers Documentation. https://www.sbert.net/

12. MongoDB (2024). GridFS Documentation. https://www.mongodb.com/docs/manual/core/gridfs/

13. Spring (2024). Spring Boot Reference Documentation. https://spring.io/projects/spring-boot

14. Render (2024). Render Deployment Documentation. https://render.com/docs

15. Hugging Face (2024). Hugging Face Spaces Documentation. https://huggingface.co/docs/hub/spaces

16. GitHub (2024). GitHub Actions Documentation. https://docs.github.com/en/actions

17. JWT.io (2024). JSON Web Token Introduction. https://jwt.io/introduction